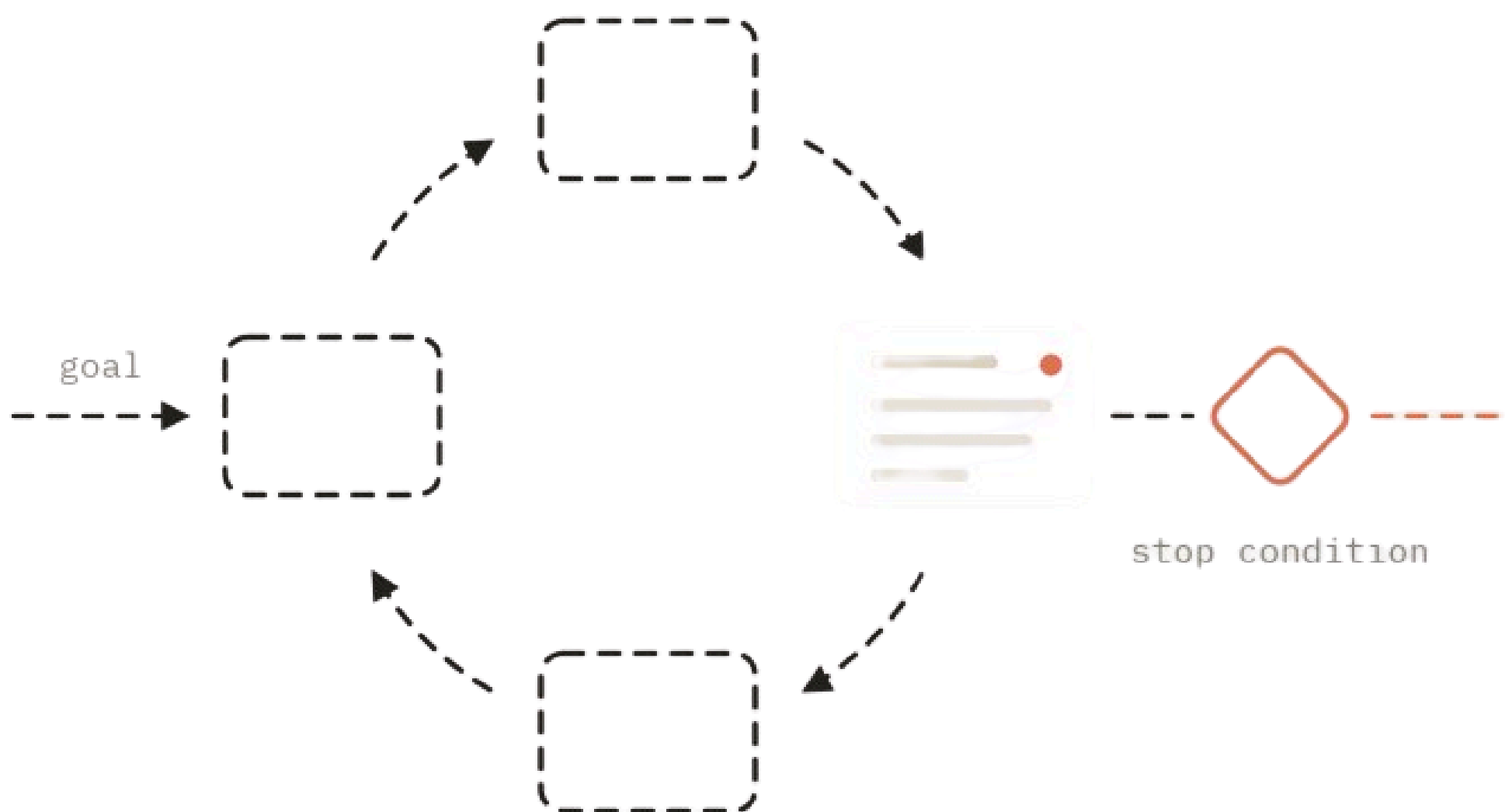


Getting started with *loops*

Different types of loops in Claude Code
and when to use them



Getting started with loops

There's a lot of talk right now about "designing loops" instead of prompting your coding agent. If you spend some time on X trying to pin down what a loop actually is, you'll come across multiple different answers.

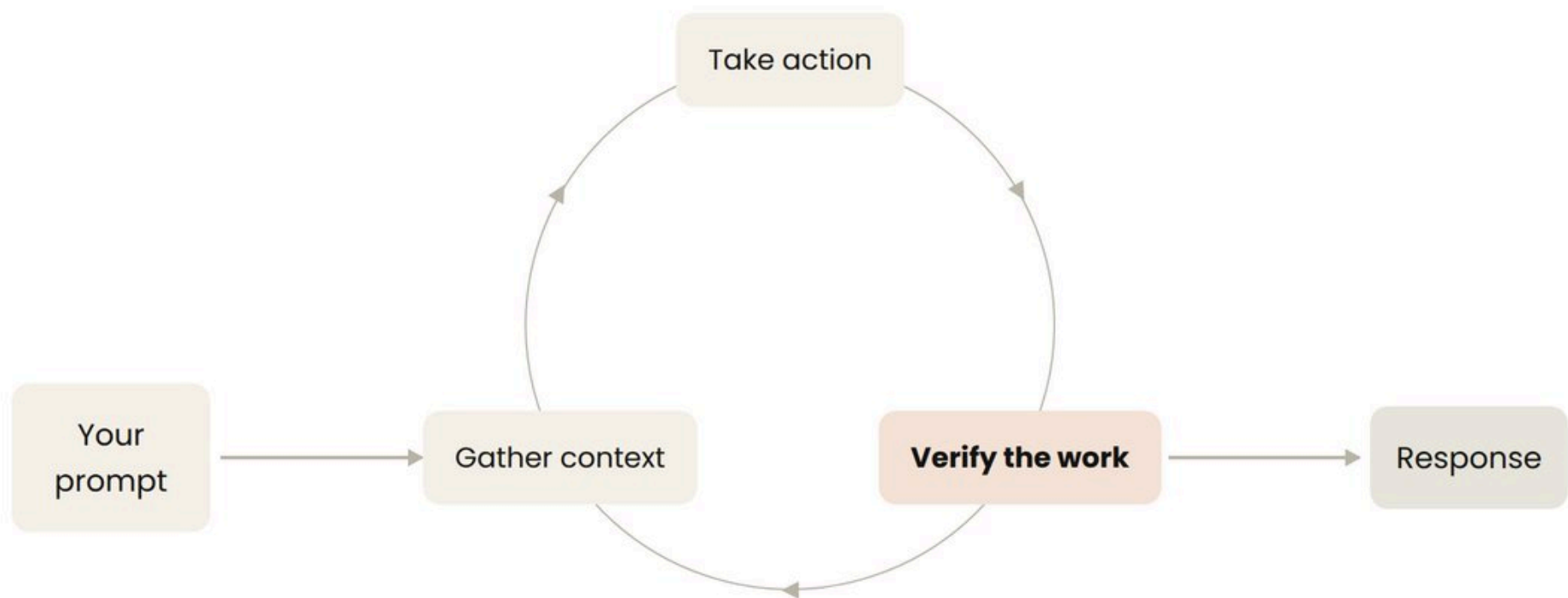
On the Claude Code team, we define loops as agents repeating cycles of work until a stop condition is met. We categorize a few different types of loops based on:

- How they are triggered
- How they are stopped
- What Claude Code primitive is used
- What type of task is most appropriate for each.

We'll cover the main loop types, when to use each, and how to maintain code quality while managing token usage.

Not all tasks require complex loops; start with the simplest solution and use these patterns selectively.

Turn-based loops



Exits when Claude judges the task complete — or the effort budget runs out.

- **Triggered by:** A user prompt.
- **Stop criteria:** Claude judges it has completed the task or needs additional context.
- **Best used for:** Shorter tasks that are not part of a regular process or schedule.
- **Managed usage by:** Write specific prompts and improve verification using skills to reduce the number of turns.

Turn-based loops

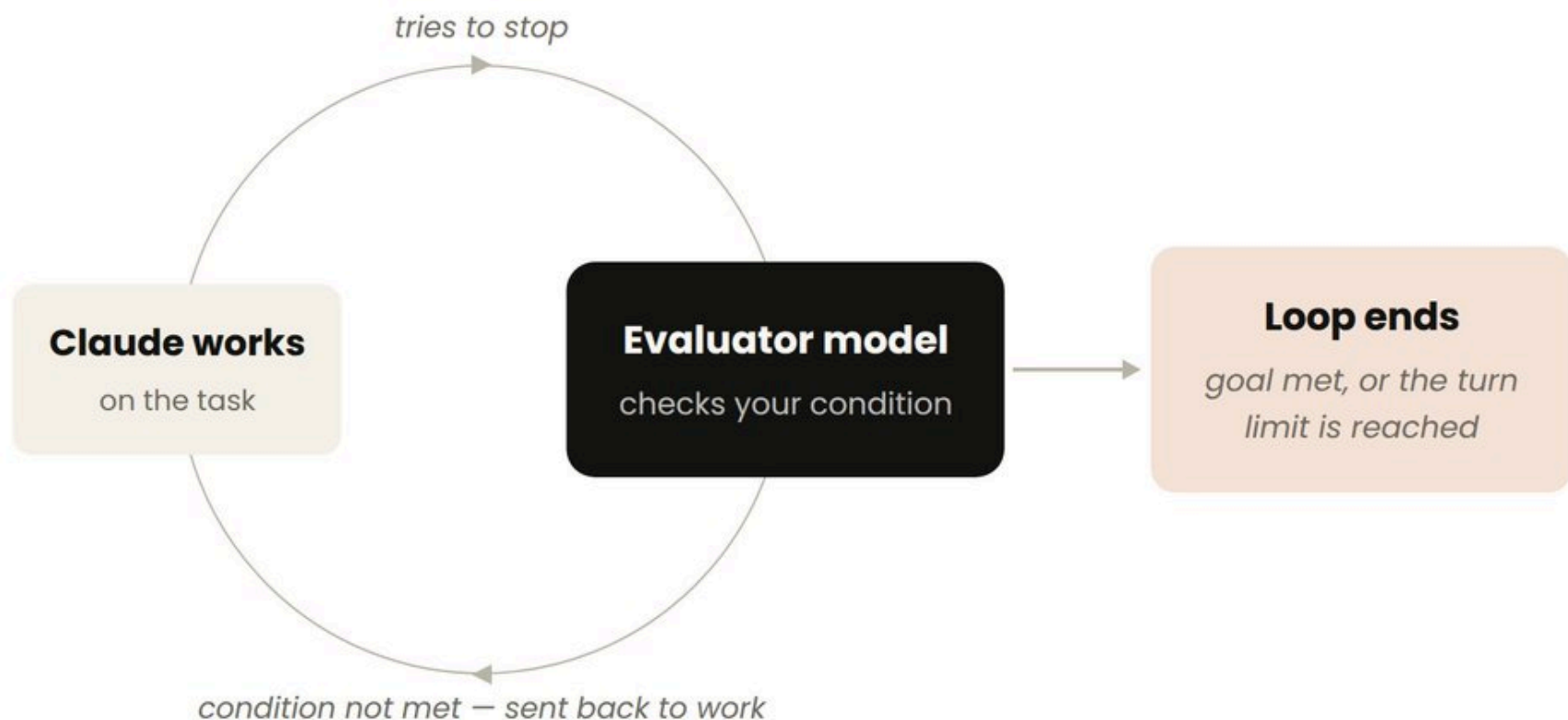
Every prompt you send starts a manual loop with you directing each turn. Claude gathers context, takes action, checks its work, repeats if needed, and responds. We call this the agentic loop.

For example, ask Claude to create a like button. It reads your code, makes the edit, runs the tests, and hands back something it believes works. You then manually check the work, and write the next prompt.

You can improve the verification step by encoding your manual steps as a SKILL.md so Claude can check more of its own work, end-to-end. This should include tools or connectors to allow Claude to see, measure or interact with the result. The more quantitative the checks are, the easier it is for Claude to self-verify.

Goal-based loop (/goal)

`/goal get the homepage Lighthouse score to 90 or above, stop after 5 tries`



- **Triggered by:** A manual prompt in real-time.
- **Stop criteria:** Goal achieved OR maximum number of turns reached.
- **Best used for:** Tasks that have verifiable exit criteria.
- **Managed usage by:** Setting a specific completion criteria and explicit turn caps, "stop after 5 tries."

Goal-based loop (/goal)

Sometimes, a single turn is not enough, especially for more complex tasks. Agents do better when they can iterate. You can extend how long Claude keeps iterating by defining what done looks like with /goal.

When you define the success criteria, Claude doesn't have to make a determination on what is "good enough" and end the loop early. Each time Claude tries to stop, an evaluator model checks your condition and sends it back to work until the goal is met or a number of turns you define is reached.

This is why deterministic criteria, such as number of tests passed or clearing a certain score threshold, are so effective.

Time-based loop

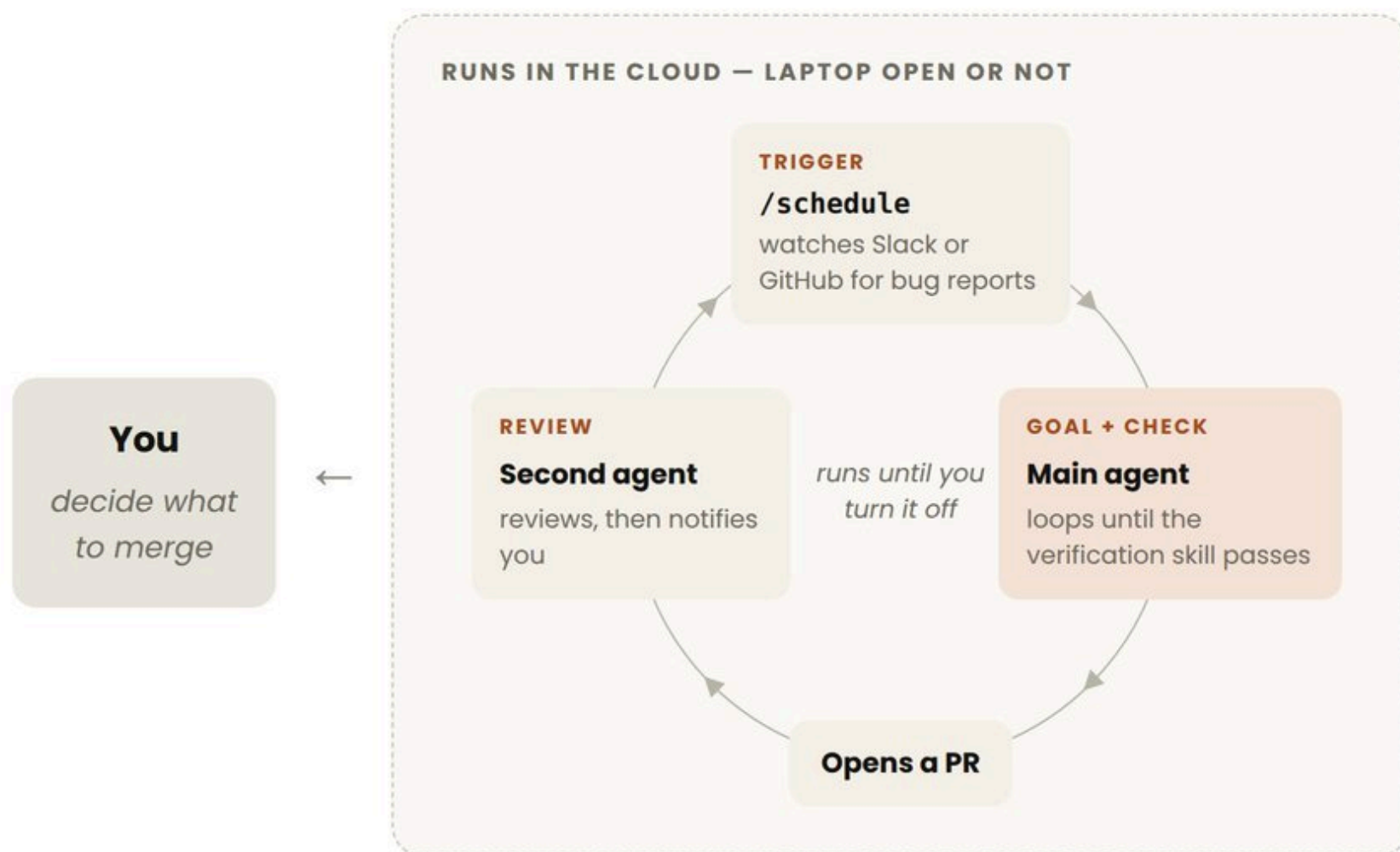
- **Triggered by:** A specified time interval.
- **Stop criteria:** You cancel it, or the work completes (the PR merges, the queue is empty).
- **Best used for:** For recurring work, or interfacing with external environments / systems.
- **Managed usage by:** Set longer intervals or react based on events rather than time.

Some agentic work is recurring: the task stays the same and only the inputs change.

For example, summarizing Slack messages every morning. Other work depends on external systems, and a simple way to interface with one is to check it on an interval and react to what changed. For example, a PR which may receive code reviews or fail CI.

For these, you can trigger when Claude runs with ``/loop`` which re-runs a prompt on an interval.

Proactive loops



- **Triggered by:** An event or schedule, with no human in real time.
- **Stop criteria:** Each task exits when its goal is met. The routine itself runs until you turn it off.
- **Best used for:** Recurring streams of well-defined work: bug reports, issue triage, migrations, dependency upgrades, etc.
- **Managed usage by:** Routing routines to smaller, faster models and using the most capable model for judgment calls.

Proactive loops

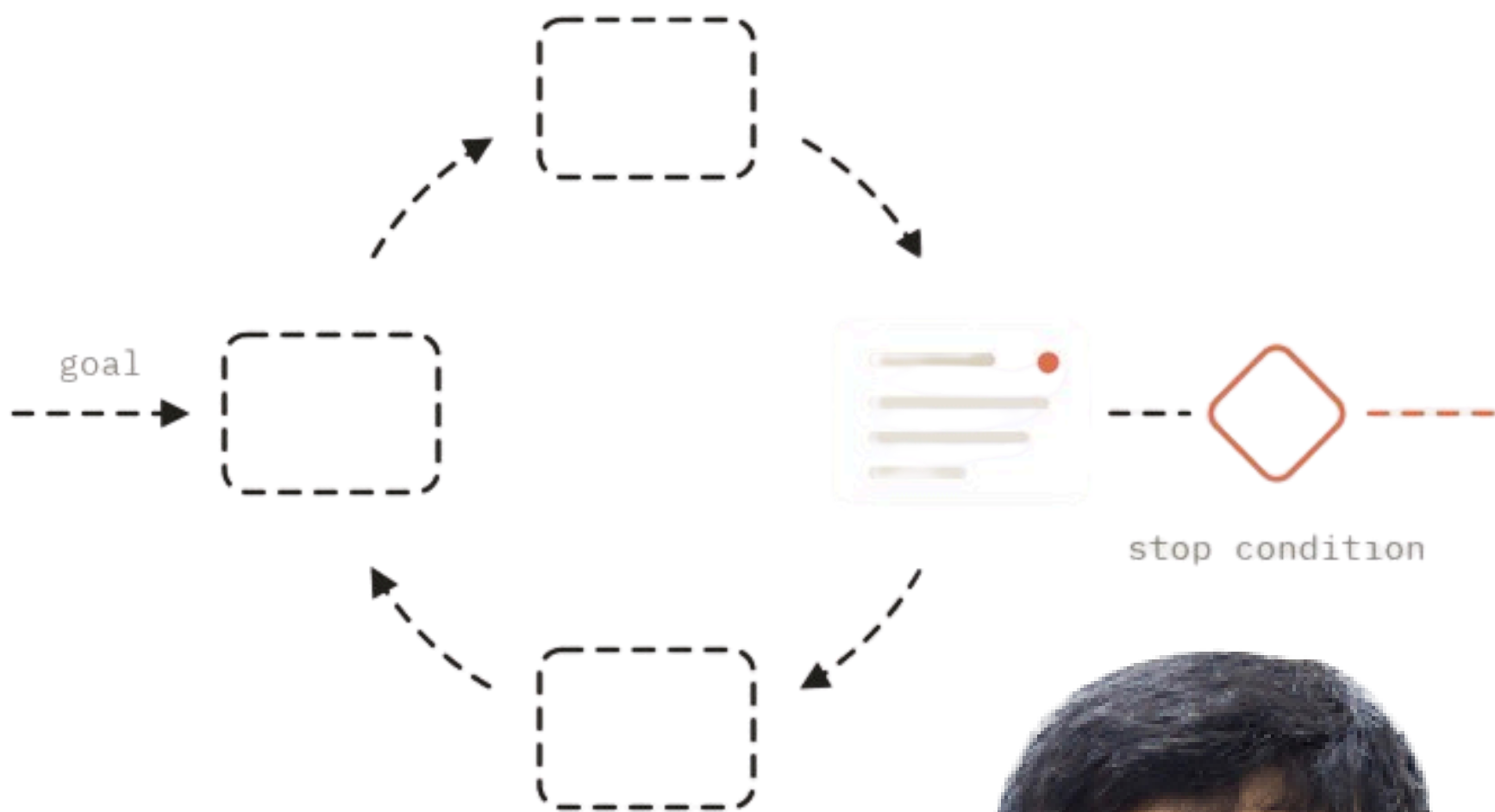
The primitives above, along with other Claude Code features like auto mode and dynamic workflows (research preview) can be composed into a loop for long-running work.

For example, to handle incoming feedback, you can use:

1. `/schedule` (research preview) to run a routine that checks for new reports`
2. `/goal` to define what done looks and skills to document how to verify it`
3. Dynamic workflows to orchestrate agents that triage each report, fix it, and review the fix
4. Auto mode so the routine runs without stopping to ask for permission

Hack Session

The Anatomy of an **Agent Loop**: From Prompt to Action to Outcome



Sudalai Rajkumar

AI Innovations

Tiger
Analytics

